

Waypoint System Architecture

Team Nova | Tech Triathlon 2026 | Hackathon submission

Waypoint coordinates Store Managers, Dispatchers, Loaders, and Drivers through a shared web application. It uses a modular monolith: one Next.js application provides the interface and backend, while PostgreSQL stores the operational records. Shared packages separate domain definitions, allocation logic, and database access within the same application.

Application and data components

- Browser client: responsive role workspaces, form interaction, status presentation, and the driver offline queue. Users access the same application from desktop, tablet, or phone.
- Next.js application: page rendering, request validation, authentication, role authorization, workflow orchestration, and the health endpoint. React Client and Server Components provide the role interfaces; Server Actions and Route Handlers process backend requests.
- Allocation package: deterministic checks for vehicle, capacity, temperature, access, depot, brand, and district constraints. The server applies these checks before accepting the relevant workflow changes.
- Domain package: shared role, label, and catalog definitions used across the application and allocation package.
- Database package: Prisma Client, schema, migrations, bootstrap, and demonstration seed. PostgreSQL 16 stores operational, audit, notification, proof, and synchronization records.

Package boundaries

The web application in `apps/web` uses `packages/domain`, `packages/allocation`, and `packages/database`. The allocation package depends on the domain package. The database package connects the application to PostgreSQL through Prisma.

There is no separate API server. Next.js provides the backend boundary, keeping deployment small while preserving server-side validation and controlled database access.

Architecture diagram

The next page shows the supplied architecture image. Its application layers are parts of one Next.js service. Railway hosts the production application and database; Docker Compose provides the local setup.

Waypoint Architecture Diagram

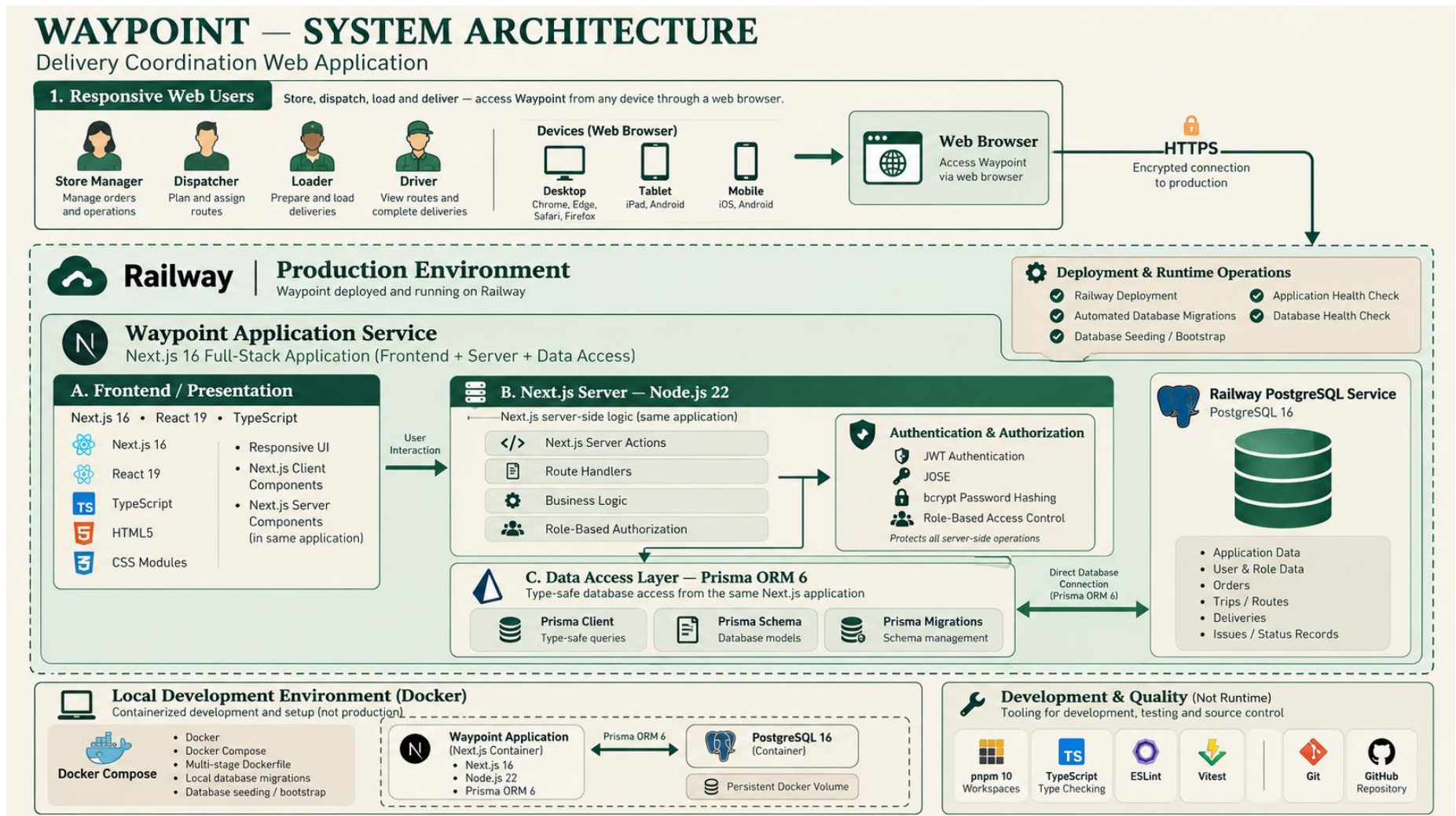


Figure 1 Four browser workspaces connect to one application service and PostgreSQL. The lower panels show local setup and development tooling. Authentication protects server operations before workflow data is read or changed.

Runtime Flow and Operational Boundaries

From user action to saved result

A role action reaches Next.js over HTTPS in production. The server verifies the signed session and role, then validates the business command and account scope. The workflow service uses Prisma to read and write PostgreSQL records. After a successful transaction, affected views are revalidated and the user receives the confirmed result. A failed operation returns a visible error.

Authorization runs before protected data is loaded or changed. Business-critical changes involving several records use transactions, so an incomplete plan, load, delivery, or receipt is not presented as successful.

Production and local deployment

The supplied architecture places the Next.js application and PostgreSQL on Railway as separate managed services. The image identifies Next.js 16, React 19, Node.js 22, and Prisma ORM 6. HTTPS terminates at managed ingress before traffic reaches the application.

For local setup, Docker Compose runs the application on container port 3000 and PostgreSQL on port 5432, connected through a private Docker network. The supplied configuration exposes the application on host port 3001 and PostgreSQL on loopback port 5433. A persistent waypoint-postgres volume retains database records. Only the TLS endpoint should be publicly accessible in production.

Reliability and consistency

- Database health checks gate local application startup. Prisma migrations run before the production server starts. The idempotent demo seed preserves an existing non-demo published plan.
- Plan publication uses serializable transactions and expected-version checks. Changes to published plans invalidate confirmed loads that need re-verification.
- Workflow success is acknowledged after persistence. Driver synchronization uses stable operation IDs and explicit pending, acknowledged, conflict, and failed states.

Security and access

- Passwords are hashed with bcrypt. Signed JWT sessions use HttpOnly cookies, expire after eight hours, and use Secure cookies in production.
- Server-side role checks protect workspaces and mutations. Store access is outlet-scoped; operational access is restricted by depot or trip.
- Environment variables supply secrets, which are excluded from Git and the Docker build context.

Supporting documentation

DATA_MODEL.pdf describes entity relationships and integrity rules. The root README provides setup instructions and the judge walkthrough. Git, pnpm, TypeScript checking, ESLint, and Vitest support development and verification; they are not separate runtime services.